Learning to Infer Competitive Relationships in Heterogeneous Networks

YANG YANG, Tsinghua University, Zhejiang University
JIE TANG and JUANZI LI, Tsinghua University

Detecting and monitoring competitors is fundamental to a company to stay ahead in the global market. Existing studies mainly focus on mining competitive relationships within a single data source, while competing information is usually distributed in multiple networks. How to discover the underlying patterns and utilize the heterogeneous knowledge to avoid biased aspects in this issue is a challenging problem. In this article, we study the problem of mining competitive relationships by learning across heterogeneous networks. We use Twitter and patent records as our data sources and statistically study the patterns behind the competitive relationships. We find that the two networks exhibit different but complementary patterns of competitions. Overall, we find that similar entities tend to be competitors, with a probability of 4 times higher than chance. On the other hand, in social network, we also find a 10 minutes phenomenon: when two entities are mentioned by the same user within 10 minutes, the likelihood of them being competitors is 25 times higher than chance. Based on the discovered patterns, we propose a novel Topical Factor Graph Model. Generally, our model defines a latent topic layer to bridge the Twitter network and patent network. It then employs a semi-supervised learning algorithm to classify the relationships between entities (e.g., companies or products). We test the proposed model on two real data sets and the experimental results validate the effectiveness of our model, with an average of +46% improvement over alternative methods. Besides, we further demonstrate the competitive relationships inferred by our proposed model can be applied in the job-hopping prediction problem by achieving an average of +10.7% improvement.

CCS Concepts: • **Information systems** → **Social networks**; *Data mining*; • **Computing methodologies** → **Artificial intelligence**;

Additional Key Words and Phrases: Social network, competitive relationship, heterogeneous network

ACM Reference format:
Yang Yang, Jie Tang, and Juanzi Li. 2018. Learning to Infer Competitive Relationships in Heterogeneous Networks. *ACM Trans. Knowl. Discov. Data.* 12, 1, Article 12 (February 2018), 23 pages.
https://doi.org/10.1145/3051127

The work is supported by the Natural Science Foundation of China (2014CB340506, 61561130160), a research fund supported by MSRA, the Royal Society-Newton Advanced Fellowship Award, and Chinese National Key Foundation Research (61533018).
Authors' addresses: Y. Yang, Room 415, CGB Building, Yuquan Campus, Zhejiang University; email: yangya@zju.edu.cn; J. Tang, Room 308, FIT Building, Tsinghua University; email: jietang@tsinghua.edu.cn; J. Li, Room 206, Eastern main building, Tsinghua University; email: lijuanzi@tsinghua.edu.cn.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2018 ACM 1556-4681/2018/02-ART12 $15.00
https://doi.org/10.1145/3051127

1 INTRODUCTION

"Competitive strategy is an area of primary concern to managers, depending critically on a subtle understanding of industries and competitors" [28]. Indeed, competition is becoming extremely fierce in every domain with companies all over the world striving for limited resources and markets. Detecting and monitoring competitors becomes a critical issue for a company to make marketing strategies. Traditional competitor detections are usually based on observations, conjectures, or sales reports. However, it is highly infeasible to manually collect the competitive relationships, considering the innumerous companies/products in the world.[1]

Recently, a few researchers have studied the problem of competitor detection. For example, Bao et al. [1] proposed an algorithm called CoMiner to identify competitors for a given entity. In this work, competitors are ranked according to the combination of several metrics including mutual information, match count, and candidate confidence. Sun et al. [30] studied the comparative web search problem, in which the user inputs a set of entities (keywords) and the system tries to find relevant and comparative information from the web for those entities. However, both works are motivated by only mining competitive relationships and not trying to reveal in which topic two entities are competing on (such as Game, Hardware, or Operation System).

In this work, we aim to conduct a systematic investigation of the problem of mining competitive relationships between entities (e.g., companies or products). Different from the related works, we try to utilize and learn from two data sources: text documents (patents) and social networks (Twitter). The reason we are using more than one data source is to avoid potential problems caused by information asymmetry. For example, some emerging companies or startups may not have any patent records. A challenge we met is how to intertwine the two sources' information properly. After all, Twitter is usually a place where the public discusses about outside apparent features yet patent records document inner core technologies that enable such features. They are entirely different in terms of contents and perspectives. Ideally, the method should combine the two pieces of information together as a heterogeneous network and thereby mine competitive relationships within it.

To clearly demonstrate the problem, Figure 1 gives an example of competitive relationships. The centered nodes are two companies: Google and Microsoft. The labels on each link indicate the fields on which the linked two companies compete with each other and the probability that connected nodes are competitors. For example, there are some well-known competitive relationships: Google competes with Facebook on social network, and competes with Microsoft on search engine. Some other competitive relationships (such as Microsoft competes with Kingsoft[2]) that are not so obvious and may be ignored by manual analysis can also be found in the figure. Such a graph of competitive relationships would be significantly helpful for a company to design market strategies. The problem is non-trivial and poses a set of challenges:

- *Multi-aspects.* A company is often associated with different topics and has different competitors on each of the fields corresponding to the topics. It is important to extract the topics and associate each competitive relationship with the topic information.
- *User generated content.* User generated content is an important source for mining entities' relationships. For example, [22] employed comparable questions to identify comparable entities. We also find a "10-minute phenomenon" from the Twitter data, as shown in Figure 4(a), if two companies are mentioned by a user in her tweet(s) in 10 minutes, there is a likelihood of 44% that the two companies are competitors, which is 25 times higher than chance. While

[1]Merely in U.S., there are more than 27 million companies, http://www.census.gov/econ/smallbus.html.
[2]Kingsoft has the second largest market share in Japan on office suite.

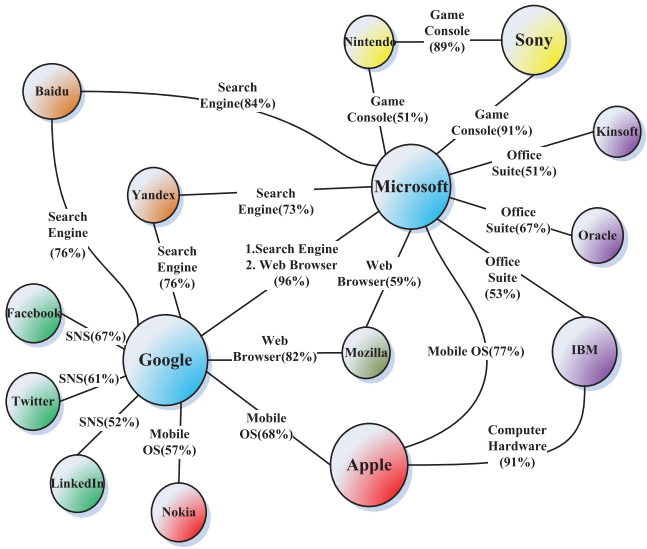


Fig. 1. Examples of topic-level competitors. Each edge is associated with fields on which the connected nodes are competitive and the probability (to which extent) they are competing on.

on the other hand, the user generated data is very unbalanced and sparse—less than 20% of the company names examined in our experiments are mentioned on Twitter.

— *Heterogeneous sources.* Patent record is another important source for mining competitive relationships, in particular on technologies. Different from the user generated content, patents contain rich, but also much irrelevant "information" such as the disclosure statement. An interesting, but challenging, question is how to combine the user generated content and the patent information together for mining competitive relationships.

In this article, we precisely define the problem of mining competitive relationships by learning across heterogeneous networks and propose a semi-supervised Topical Factor Graph Model (TFGM). An efficient algorithm is developed to learn the proposed model. We evaluate the proposed model on a large patent network and the Twitter network. Experimental results demonstrate that the proposed model can extensively improve the performance (averagely +46% in terms of F1-Measure) over several alternative methods. At last, we demonstrate how to apply the inferred competitive relationships between entities. In this work, we use the job-hopping prediction problem as an example. To summarize, we have the following findings through this study:

— Social network information is important for competitor mining. Actually, merely based on companies' attributes on Twitter, we can obtain a better performance (+6–57%) for mining competitive relationships than only mining on the patent data.
— Learning by utilizing heterogeneous networks can significantly improve the mining performance (+17–45%) comparing with learning over only a single network.
— It is intriguing that our experiments offer some empirical evidences for the theory of social balance [10]: "My enemy's enemy is my friend." We find a high degree (more than 90%) of balanced triads in the competitive network.
— The inferred competitive relationships between companies can be applied to the job-hopping prediction to improve the performance +10.7% in average.

Organization. Section 2 formulates the problem. Section 3 introduces the datasets and some observations we discovered. Section 4 explains our proposed model and describes the algorithm for model learning. Section 5 demonstrate how to apply the proposed model into a job-hopping prediction problem. Section 6 introduces our experiment that validates the effectiveness of our methodology, including its setup, baseline methods and results. Finally, Section 7 reviews some previous works related to ours and Section 8 concludes this work.

2 PROBLEM DEFINITION

We introduce some necessary definitions and then formulate the problem. To keep things concrete, we will use company as the example to explain the competitive relationship mining problem. The problem can be easily generalized to other entities such as products.

We consider two heterogeneous data sources: Patent and Twitter. From patent records, we extract companies, inventors, and patents. We create a network of companies $G = (V, E, \mathbf{S}), E \subseteq V \times V$, where V represents a set of companies, E represents the relationship between companies, and $\mathbf{S}$ is a matrix describing attributes associated with the companies, in which every row corresponds to a vector of attribute values of a company. For example, the attributes of a company can be inventors of those patents owned by the company and keywords occurring in the patent descriptions. Moreover, we augment the company network with social networking information. Specifically, we consider Twitter users who have discussed the companies and tweets that have mentioned the company names. Thus, the augmented network is represented as $G = (V, E, \mathbf{S}, \mathbf{U}, \mathbf{M})$, with each row of matrix $\mathbf{U}$ denoting users who have posted tweets containing the corresponding company name and each row of matrix $\mathbf{M}$ denoting tweets which contain the corresponding company name. As a conclusion, $\mathbf{S}$ is correlated to the text document (patent) data source. $\mathbf{U}$ and $\mathbf{M}$ are correlated to the social network (Twitter) data source. We further assume that each company is associated with a topic distribution. In particular, we have the following definition:

Definition 2.1 (Topic model of company). A topic model θ_d of a patent d is a multinomial distribution of words $\{P(w|\theta_d)\}$. Then a company v_i is considered as a mixture of topic models, denoted as θ_{v_i}, extracted from those patents owned by the company.

The underlying assumption for the topic model is that words appearing in the patents are sampled from a distribution corresponding to each topic, i.e., $P(w|\theta_d)$. Thus, words with the highest probabilities associated with each topic would suggest the semanteme represented by the topic. For example, a "Search Engine" topic can be represented by keywords "search," "advertisement," and "ranking."

For each edge $e \in E$, we associate it with a label $y \in \{0, 1\}$. $y = 1$ indicates corresponding two companies have a competitive relationship. Given that, we can define the problem addressed in this article:

Problem 1 (Competitive Relationship Mining). *Given a network, $G = (V, E, \boldsymbol{S}, \boldsymbol{U}, \boldsymbol{M})$ and topic models $\{\theta\}$ of all companies, the goal is to learn a predictive function $f : (E|G) \rightarrow Y$ to infer the competitive label of each relationship between companies.*

There are two things worth mentioning. The first is in the network G, we may have some labeled data, i.e., labeled competitive relationships from some online databases, but for most of the relationships the labels are unknown. The second is that the network is theoretically a complete network. We could use some parameters or human knowledge to control the density of the network. For example, only when the similarity of two companies (based on content or network information) is larger than a predefined threshold, we add an edge between them.

Another interesting problem is, after mining the competitive relationships between companies at time t, predicting the occurrence of job-hopping between two companies at time $t+1$. For a pair of companies A and B, we use $l^{(A,B)} = 0$ to denote there will be no job-hopping happen at time $t+1$; $l^{(A,B)} = 1$ denotes there will be more people "hop" from A to B than opposite direction; $l^{(A,B)} = -1$ denotes there will be more people "hop" from B to A. Formally, we define this problem as follows:

Problem 2 (Job-hopping Prediction). *At a specific time t, given a network of companies at t, $G^t = (V^t, E^t, \boldsymbol{S}^t)$, where V^t and E^t are sets of companies and their relationships existing at time t, $\boldsymbol{S}^t$ is a matrix describing attributes associated with the companies at time t, also given the topic models $\{\theta\}^t$, which are extracted from patents authorized within time t, of all companies, the competitive label of each relationship between companies, Y^t, and a pair of two companies (A, B), the goal is to learn a function $g : (A, B|G^t, \{\theta\}^t, Y^t) \rightarrow l^{(A,B)}$.*

3 DATA AND OBSERVATION

Before presenting our approach for competitor detection, we first convey a series of discoveries we observed from the data.

3.1 Data Collection

In this study, we consider two data sources: Patent and Twitter. We have collected all the patents (3,770,411 patents) from USPTO,[3] from which we extracted 195,263 companies and 2,430,375 inventors. For each company, we used it as the query to search Twitter and retrieved the top returned tweets, from which we further extracted the information of users. So far, we have collected 1,033,750 tweets written by 87,603 Twitter users, which cover 1,393 major companies. In looking for benchmark data, we turn to *Yahoo! Finance*[4] and use it as the ground truth source.[5] Each company name was sent as a query to obtain its competitor list.

The probability of two randomly picked companies being competitors among the whole dataset (1.59%, testified) is assigned to be the baseline probability. We compare our observation with it in order to see how different features affect the competitive relationship between two companies.

3.2 Observations

We conduct our observation analysis to answer the following three questions: (1) How likely two companies are competitors if they have similar business? (2) How likely two companies are competitors if they are often discussed in social medias? (3) Whether the phenomenon of "my enemy's enemy is my friend" exists in the competitive network or not?

Patent Analysis. Social theory homophily suggests that similar individuals tend to associate with each other [20]. Here, we show how similarity degree of two companies correlates with the competitive relationship between them. We consider two types of similarities. The first one is based on words occurring in the descriptions of patents owned by the two companies. The second one is based on the number of common inventors, i.e., inventors that used to work for both companies at different times. For the former, we respectively generate two topic distributions θ_{v_i} and θ_{v_j} of the two companies v_i and v_j by PLSA [11] (see Section 4 for details). The similarity between the

[3] http://www.uspto.gov/.
[4] http://finance.yahoo.com/.
[5] For example, IBM's competitors can be found at this page: http://finance.yahoo.com/q/co?s=IBM+Competitors.

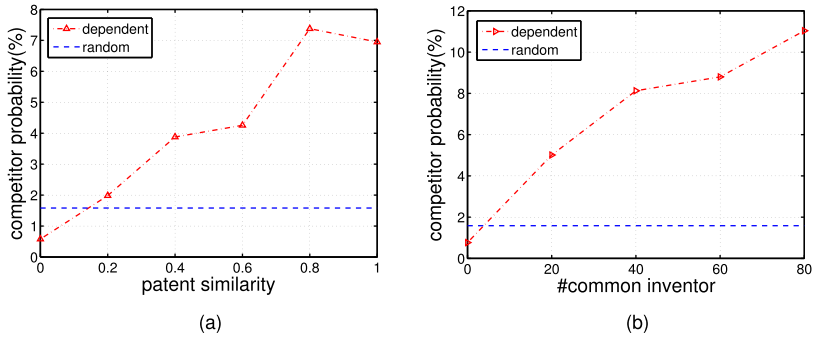


Fig. 2. (a) Patent similarity correlation. *X*-axis: patent similarity between two companies. (b) Common employed inventors correlation. *X*-axis: the number of common inventors of two companies.

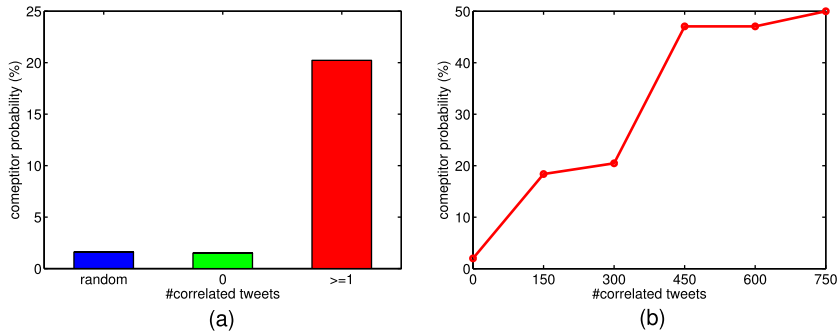


Fig. 3. Tweet-level analysis. *Y*-axis: the probability of two companies being competitors, conditioned on the number of their co-occurring tweets.

two companies is calculated by cosine similarity:

$$Sim(v_i, v_j) = \frac{\theta_{v_i} \cdot \theta_{v_j}}{||\theta_{v_i}||||\theta_{v_j}||}. \tag{1}$$

Figure 2(a) clearly shows that, when the similarity of two companies increases from zero, the likelihood of them being competitors rapidly increases and becomes four times the likelihood of two random companies. We observe a similar pattern for the analysis on inventors as shown in Figure 2(b). When no common employed inventors can be found from two companies, the probability of them being competitors drops to 1.32%, lower than the baseline probability. However, with more common inventors being detected, the probability outnumbers the baseline data and keeps increasing.

Twitter Analysis. We study the likelihood of two companies being competitors when their names co-occur in tweets. Figure 3 shows the analysis results. It is striking that when the names of two companies are mentioned together in one tweet, the likelihood of the two companies being competitors becomes more than 10 times higher than chance. Figure 3(b) further demonstrates that the likelihood will continue to increase when the number of co-occurring tweets increase.

Besides the tweet-level co-occurrence, we conduct another analysis on the user-level. Figure 4(a) shows that when a user mentions two companies in 10 minutes (may in different tweets), the likelihood of the two companies being competitors is 25 times higher than chance. Figure 4(b)

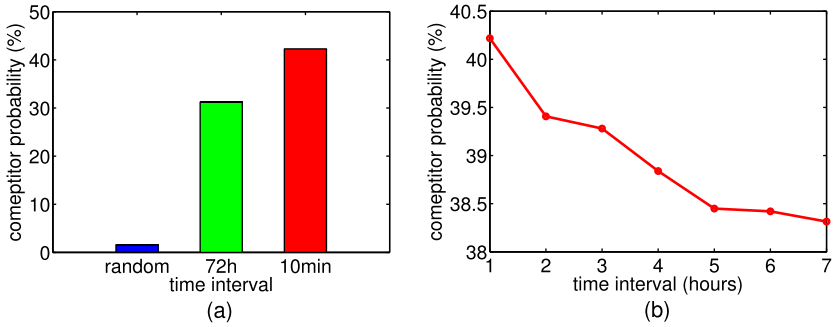


Fig. 4. User-level analysis. *Y*-axis: the probability of two companies being competitors, conditioned on a user mentions the two companies within a particular time interval.

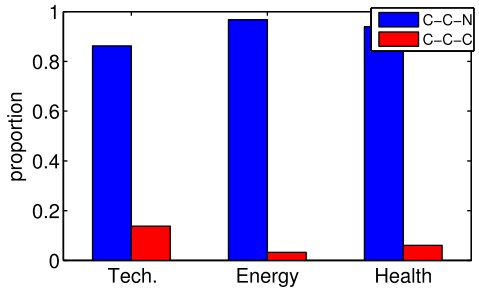


Fig. 5. Comparison of two triad relationships. C–C–N: competitor–competitor–non-competitor (balanced); C–C–C: competitor–competitor–competitor (unbalanced). *Y*-axis: the proportion of the balanced and unbalanced triad relationships.

further illustrates that the likelihood drops when we set the time interval larger. We suppose that since tweets have length limitation, when a user discusses a company or its product in one tweet, she may follow up with another to mention its competitors or competitors' products.

Is my enemy's enemy again my enemy? We study whether competitors form a balanced network structure. The phenomenon of "the enemy of my enemy is my friend" is one of the underlying balanced triad suggested by the social balance theory [10].

In particular, we split the data into three domains: Tech. (technology), Energy and Health. In each domain, companies are grouped in triads. We use $e_{ij} = 1$ to represent that company v_i and v_j are competitors, and use $e_{ij} = 0$ to denote there is no competitive relationship between v_i and v_j. Given a triad (v_i, v_j, v_k), we compare the likelihood of $(e_{ij} = 1 \wedge e_{jk} = 1) \Rightarrow e_{ik} = 0$ (denoted as C–C–N) and that of $(e_{ij} = 1 \wedge e_{jk} = 1) \Rightarrow e_{ik} = 1$ (denoted as C–C–C). Figure 5 shows the probability of balanced triad in the three domains, from which we could have the following summary: my enemy's enemy is not necessary my friend, but can hardly be my enemy again. This may be caused by the fact that two companies tend to union when they have a common enemy to compete against.

To sum up, according to the statistics shown above, we have the following discoveries:

(1) As expected, similar companies tend to be competitors, with a probability of four times higher than chance.
(2) Social network information is a very important indicator for competitors. The likelihood of two companies being competitors is 10 times higher than chance when they are

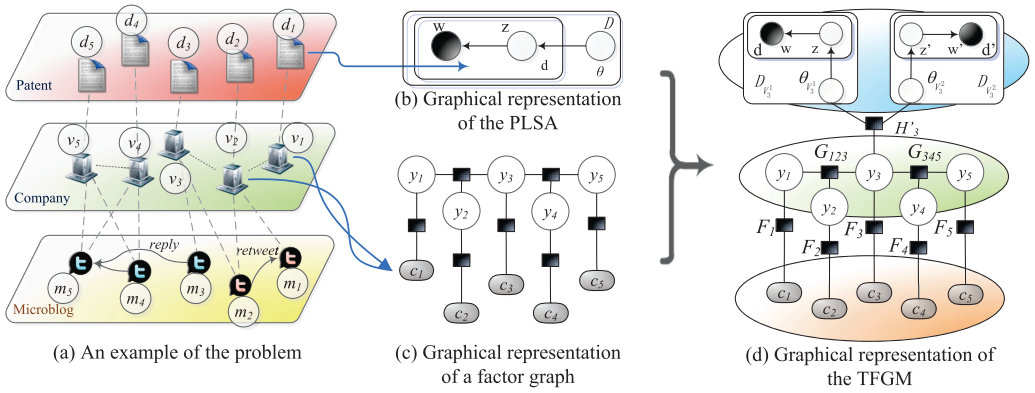


Fig. 6. An example of the problem and graphical representations of three different models.

mentioned in a same tweet and increases to 25 times when they are mentioned by the same user within 10 minutes.

(3) My enemy's enemy is not necessary my friend, but should not be my enemy again (with a 90% likelihood).

4 TOPICAL FACTOR GRAPH MODEL

4.1 Model Description

In this section, we propose a novel model referred to as TFGM for mining competitive relationships. As we mentioned in Section 3, entities that have similar topic distributions are more likely to be competitors and vice versa, competitors tend to have similar topic distributions. Thus, the basic idea of the proposed model is to combine factor graph and topic model together, and learn them simultaneously. Before we present the proposed model, we first briefly discuss two basic models: topic models and factor graphs.

Topic model. Statistical topic models have been successfully applied to many text mining tasks [2, 11]. The basic idea of these models is to model documents with a finite mixture model of K topics and estimate model parameters by fitting a dataset with the model. Two basic statistical topic models are Probabilistic Latent Semantic Analysis (PLSA) [11] and Latent Dirichlet Allocation (LDA) [2]. For example, the log likelihood of a collection D to be generated with PLSA is given as follows:

$$\log p(D) = \sum_{d} \sum_{w} n(w,d) \sum_{j=1}^{k} \log[p(w|z_j)p(z_j|d)], \tag{2}$$

where $n(w,d)$ denotes the occurrences of word w in a text document d, z_j is a topic and the parameters to estimate in PLSA model are $p(w|z_j)$ and $p(z_j|d)$ (or θ_d). An example of PLSA's graphical representation is shown in Figure 6(b). θ in the figure stands for the topic distribution of each text document in the dataset. Given this, we can define the topic distribution of each vertex (or entity, e.g., company, product) v_i in G as a mixture of topic distribution over text documents (e.g., patents) D_{v_i} associated with v_i, i.e.,

$$\theta_{v_i} = p(z_j|v_i) = \sum_{d \in D_{v_i}} p(z_j|d)p(d|v_i) = \sum_{d \in D_{v_i}} \frac{p(z_j|d)}{|D_{v_i}|}. \tag{3}$$

Factor graph. A factor graph consists of two layers of nodes, i.e., variable nodes and factor nodes, with links between them. The joint distribution over the whole set of variables can be factorized as a product of all factors. A factor graph can be learned via some efficient algorithms like the sum-product algorithm [18].

Figure 6(c) gives an example of modeling our problem with the factor graph, which incorporates entity pairs' information and labels of their relationships. For each pair of entities (v_i, v_j), we create an instance node c_k in the factor graph. For easy explanation, we use c_k^1 and c_k^2 to denote v_i and v_j, respectively. The hidden variable y_k stands for the label of the relationship, with $y_k = 1$ indicating c_k^1 and c_k^2 have a competitive relationship, $y_k = 0$ not, and $y_k =?$ unknown. Our objective in the factor graph is to assign a value to the unknown y_k with high accuracy.

Topical Factor Graph Model. We formally present the proposed TFGM, which leverages the power of the two basic models and formulates the competitor detection problem in a unified learning framework.

Given a network $G = (V, E, \mathbf{S}, \mathbf{U}, \mathbf{M})$ with some labeled relationships Y, our objective can be formalized as to maximize the following posterior probability:

$$p(Y|G) \propto p(D|\Theta)p(Y|G, D, \Theta), \tag{4}$$

where D is a collection of all text documents. The first term on the right side of Equation (4) can be defined according to the topic model and the second term can be defined as a factor graph. Further, to incorporate the intuition that competitors tend to have similar topic distributions, we define a regularizer, which is similar to the graph harmonic function in [44], to quantify the difference between topic distributions of two entities:

$$R(Y, \Theta) = \frac{1}{2} \sum_{y_i=1} \sum_{j=1}^{K} ||\theta_{c_i^1 j} - \theta_{c_i^2 j}||^2, \tag{5}$$

where K is the total number of topics.

By integrating Equations (4) and (5) together, we can define the following objective function to our problem:

$$O(G) = (1 - \lambda) \log p(D|\Theta)p(Y|G, D, \Theta) - \lambda R(Y, \Theta) \tag{6}$$

where λ is a parameter to balance the importance of the two terms.

Now we discuss how to instantiate the objective function. We can use any statistical topic model to define $p(D|\Theta)$. In this article, we use PLSA. As to formalize $p(Y|G, D, \Theta)$, we study the corresponding entities' correlation and attributes, and we define the following three factors according to the intuitions we have discussed.

- *Attribute factor:* $F(\mathbf{x_i}, y_i)$ represents the posterior probability of y_i given the attribute vector $\mathbf{x}_i$, where $\mathbf{x}_i = (\mathbf{x}_i^1, \mathbf{x}_i^2)$, $\mathbf{x}_i^1 = (\mathbf{S}_{c_i^1}, \mathbf{U}_{c_i^1}, \mathbf{M}_{c_i^1})$, and $\mathbf{x}_i^2$ is defined similarly.
- *Balanced triangle factor:* $G(Y_c)$ reflects the correlations between each clique in Y. A set of three label nodes Y_c is a clique if the nodes stand for relationships between three entities.
- *Topic factor:* $H(y_i, \theta_{c_i^1}, \theta_{c_i^2})$ denotes the posterior probability of y_i given two corresponding entities' topic distribution.

Jointing the factors defined above, we have

$$p(Y|G, D, \Theta) = \prod_i F(\mathbf{x_i}, y_i)H(y_i, \theta_{c_i^1}, \theta_{c_i^2}) \prod_c G(Y_c), \tag{7}$$

where Y_c is a triad derived from the input network. The three factors can be instantiated in different ways. In this work, we use exponential-linear functions. In particular, we define the three factors

as follows:

$$F(\mathbf{x_i}, y_i) = \frac{1}{Z_1} \exp\left\{\sum_{j=1}^{|x_i|} \alpha_j f_j(x_{ij}, y_i)\right\}, \tag{8}$$

$$G(Y_c) = \frac{1}{Z_2} \exp\{\beta g(Y_c)\}, \tag{9}$$

$$H(y_i, \theta_{c_i^1}, \theta_{c_i^2}) = \frac{1}{Z_3} \exp\{\gamma h(y_i, \theta_{c_i^1}, \theta_{c_i^2})\}, \tag{10}$$

where Z_1, Z_2, and Z_3 are normalization factors. $f_j(x_{ij}, y_i)$ and $h(y_i, \theta_{v_i^1}, \theta_{v_i^2})$ can be defined as either a binary function or real-valued function. $g(Y_c)$ can be defined as an indicator function.

Finally, by plugging Equations (2) and (7–10) into Equation (6), we have

$$\begin{aligned} O(\Psi) = (1-\lambda) \Bigg[& \sum_d \sum_w n(w,d) \log \sum_{j=1}^{k} p(w|z_j) p(z_j|d) \\ & + \sum_{i=1}^{|Y|} \sum_{j=1}^{d} \alpha_j f_j(x_{ij}, y_i) + \sum_c \beta g(Y_c) \\ & + \sum_{i=1}^{|Y|} \gamma h(y_i, \theta_{c_i^1}, \theta_{c_i^2}) - \log Z \Bigg] - \lambda R(Y, \Theta), \end{aligned} \tag{11}$$

where Ψ is the collection of parameters, i.e., $\Psi = \{p(w|z_j)\} \cup \{p(z_j|d)\} \cup \{\alpha_i\} \cup \{\beta\} \cup \{\gamma\}$, and $Z = Z_1 Z_2 Z_3$ is a normalization factor. Our goal is to estimate a parameter configuration Ψ to maximize the objective function $O(\Psi)$.

The graphical representation of TFGM is shown in Figure 6(d). The upper layer is used for modeling the topic extraction task and the bottom layer is designed to model the competitor detection task. Actually we can combine $R(Y, \Theta)$ and $H(Y, \Theta)$ together as one factor function H' to bridge the two tasks. We separate $R(Y, \Theta)$ and $H(Y, \Theta)$ to easily explain how we learn the model in the rest of this section.

4.2 Model Learning

To estimate the parameters in TFGM, let us first consider the special case when $\lambda = 0$. The objective function degenerates to $\log p(Y|G)$ with no regular function in this case. To maximize $\log p(Y|G)$, we first apply an Expectation Maximization (EM) algorithm, a standard way of parameter estimation of PLSA, to iteratively compute a local maximum of $\log p(D|\Theta)$. After that, we compute the values of Θ based on Equation (3) and maximize $\log p(Y|G, D, \Theta)$ by a gradient descent method. We repeat the two steps until the objective function converges.

The details of how to estimate the parameters of PLSA can be seen in [11]. When computing $p(Y|G, D, \Theta)$, we need to sum up the likelihood of possible states for all the nodes, including the unlabeled ones, to normalize Z. To deal with this, we infer the unlabeled labels from known ones. Y^U is denoted as a labeling configuration inferred from known labels. We then have

$$\begin{aligned} \log p(Y|G, D, \Theta) &= \log \Sigma_{Y^U} p(Y^U|G, D, \Theta) \\ &= \log \Sigma_{Y^U} \exp\{\mu^T \mathbf{Q}(Y^U)\} \\ &\quad - \log \Sigma_Y \exp\{\mu^T \mathbf{Q}(Y)\}, \end{aligned} \tag{12}$$

where $\mathbf{Q}(Y) = ((\sum_i f_j(x_{ij}, y_i))^T, \sum_c g(Y_c), \sum_i h(y_i, \theta_{c_i^1}, \theta_{c_i^2}))^T$, and $\mu = (\alpha^T, \beta, \gamma)^T$.

We introduce the gradient descent method to solve the function. The gradient for each parameter μ is calculated as

$$\begin{aligned}\nabla &= \frac{\partial \log p(Y|G, D, \Theta)}{\partial \mu} \\ &= \mathbb{E}_{p_\mu(Y^U|G,D,\Theta)}\mathbf{Q}(Y^U) - \mathbb{E}_{p_\mu(Y|G,D,\Theta)}\mathbf{Q}(Y),\end{aligned} \tag{13}$$

One challenge here is to directly calculate the two expectations. The graphical structure of TFGM may be arbitrary and contain cycles. Thus, we adopt Loopy Belief Propagation (LBP) [27] approximate algorithm to compute the marginal probabilities of Y and Y^U. We are then able to obtain the gradient by summing over all the label nodes. An important point here is that the LBP process needs to be proceeded twice during the learning procedure, one for estimating $p(Y|G, D, \Theta)$ and the other for $p(Y^U|G, D, \Theta)$. We update each parameter with a learning rate ξ with the gradient.

We now discuss the case when $\lambda \neq 0$. In this general case the objective function does not have a closed-form solution. Here, we propose a simple and efficient algorithm which primarily consists of two steps. In the first step, we update $p(z_j|d)$, $p(w|z_j)$, and μ according to the same method in case $\lambda = 0$. In the second step, we fix $p(w|z_j)$ and μ to update $p(z_j|d)$ as follows:

$$p_{n+1}(z_j|d_{v_i}) = (1-\eta)p_n(z_j|d_{v_i}) + \eta\frac{\sum_{y_{(v_i,v_k)}=1}\sum_{d_{v_k}\in D_{v_k}} p_n(z_j|d_{v_k})}{\sum_{y_{(v_i,v_k)}=1}|D_{v_k}|},$$

where D_{v_k} denotes the text documents associated with v_k, $d_{v_i} \in D_{v_i}$, and $y_{(v_i,v_k)}$ stands for the label correlated with entities v_i and v_k. Clearly, $\sum_j p_n(z_j|d_{v_i}) = 1$ and $p_n(z_j|d_{v_i}) > 0$ always hold in Equation (14). When the step parameter η is set to 1, it means the new topic distribution of a text document, which belongs to entity v_i, is the average of the old distributions from all documents of v_i's competitors. This is related to the random-walk interpretation. A similar algorithm was also used in [26]. See details in Algorithm 1.

In factor graph, we can also consider making use of topic model's results to help mining competitive relationships; however, the topics are treated equally including ones that might be irrelevant to competitions. In contrast, TFGM, with the regularizer, can distinguish "competition topics" from irrelevant topics thus to mine competitive relationships more effectively.

4.3 Factor Function Definition

We introduce how we define the factor functions in our model. For *attribute factor function*, we define three categories of features.

Social Correlation. In the company dataset, we consider tweets related to both companies in two features: the number of tweets with their co-occurrence and the number of tweet-pairs published by one user in a small time interval, in which one tweet is related to one company, respectively. In the product dataset, we also consider the two similar features corresponding to reviews on products.

Social Homophily. Whether two companies or products have equal social status. In the company dataset, we define three features: the number of tweets related to each company, the number of company's official account's Twitter followers, and the number of users who follow both companies. We define two features in the product dataset: the price difference of the two products and the number of reviews on each product.

Local Homophily. In the company dataset, we extract patent and inventor information of each company and consider whether two companies have common points in this. We use two features:

ALGORITHM 1: Learning Algorithm of TFGM.

Input: a network G, a partially labeled competitor matrix Y, the learning rate η and ξ, maximum iteration number I and $\mathcal{J}$.

Output: estimated parameter Ψ

Initialize $p(z_j|d), p(w|z_j)$ randomly;
Initialize $\mu \leftarrow 0$;
repeat
  Update $p(z_j|w,d)$, $p(w|z_j)$, and $p(z_j|d)$ to maximize $\log p(D|\Theta)$ with EM algorithm.
  Calculate Θ with Equation (3);
  for $i = 1$ *to* I **do**
    Call LBP to calculate $\mathbb{E}_{p_\mu(Y^U|G,D,\Theta)}\mathbf{Q}(Y^U)$;
    Call LBP to calculate $\mathbb{E}_{p_\mu(Y|G,D,\Theta)}\mathbf{Q}(Y)$;
    Calculate ∇_μ with Equation (13);
    Update $\mu_{new} = \mu_{old} - \xi \cdot \nabla_\mu$
  end
  for $n = 1$ *to* J **do**
    Update $p_{n+1}(z_j|d_{v_i})$ with Equation (14);
  end
until *Convergence*;

the number of common inventors and the number of patents they have. In the product dataset, we consider only one feature: the number of users who reviewed both products.

For *balanced triangle factor function*, we define eight features to capture all the possible situations for every three links. We define *topic factor function* as the cosine similarity between the two topic distributions.

5 APPLICATION: JOB-HOPPING PREDICTION

Given two companies A and B, knowing whether they are competing with each other or not, we further study whether there will be a job-hopping trend occurs between A and B. See the formal definition of the job-hopping prediction problem in Section 2. To the best of our knowledge, we are the first to give a formal definition and solution of the job-hopping prediction problem.

In this section, we present our approach for job-hopping prediction. Our basic idea is first capturing the fundamental cause of job-hopping, then using them as features in a machine learning framework. Both quantitative and qualitative experiments on these models show the effectiveness of our proposed features (see details in Section 6).

Features. We begin with the features used to solve the job-hopping prediction problem. In this work, we mainly consider two kinds of features: *intellectual capital* and *social relationships*.

Intellectual capital represents the benefits new employees bring with for a company [15]. In economics, intellectual capital consists of *human capital*, which is the employees of a company who contribute through their skills and expertise, and *structural capital*, which includes the supportive infrastructure, processes, and databases of the organization that enable human capital to function (e.g., patents, buildings, hardware) [24]. Specifically, in this work, we use the number of inventors and patents of a company to represent its human capital and structural capital, respectively. Thus, given a pair of two companies, we define the intellectual capital feature as the difference of the number of two companies' inventors and patents.

We further define the social relationship feature, which includes the competitive relationships detected by our proposed model, and the hidden topic relationship between their business or research directions. More specifically, given two companies c_1 and c_2, we use a binary value to denote whether two companies are competitors or not, and define the score of their hidden topic relationship at time t as a K-dimension vector

$$h(c_1, c_2)_k = \frac{p(z_k|c_1, t) - p(z_k|c_1, t-1)}{p(z_k|c_2, t) - p(z_k|c_2, t-1)}, \tag{14}$$

where $p(z_k|c, t)$ indicates the proportion of topic z_k in company c at time t. For implementation, $p(z_k|c, t)$ is calculated by topic modeling methodologies (e.g., LDA [2]) based on patents of company c at time t. As the equation above suggests, companies with similar raising/falling trend on topic z_k would have a greater value on kth dimension.

Learning Methodology. The features described above could be used in several machine learning frameworks. Here, we use a multi-nominal logistic regression as an example to model the job-hopping probability from company c_1 to c_2 or the opposite (i.e., $I^{(c_1,c_2)}$). For simple representation, given a company c, we use matrix S_c^t to denote its *intellectual capital features* as we introduced above. We also given by the two companies' hidden topic relationship $h(c_1, c_2)$ (see details in Equation 14) and their competitive relationship y_{c_1,c_2} determined by TFGM. Based on these information, we formally define the multi-nominal logistic regression as follows:

$$P(l^{(c_1,c_2)} = l|S_{c_1}, S_{c_2}, \theta_{c_1}, \theta_{c_2}, y_{c_1,c_2}) = \frac{\exp(w_l \cdot < S_{c_1} - S_{c_2}, h(c_1, c_2), y_{c_1,c_2} >)}{1 + \sum_{l'} \exp(w_{l'} \cdot < S_{c_1} - S_{c_2}, h(c_1, c_2), y_{c_1,c_2} >)}, \tag{15}$$

where $\mathbf{w}$, a $|S_c| + 2$ dimensioned vector, is the parameter we aim to learn. Given a train dataset $\{l\}$, the objective function is defined as

$$L(\mathbf{w}, G, Y, \theta, \{l\}) = \prod_{l \in \{l\}} P(l|G, Y, \theta, \mathbf{w}). \tag{16}$$

Several classic learning algorithms like gradient descent, Newton method, and quasi Newton method could be employed to estimate the value of $\mathbf{w}$. In this work, we utilize Newton method.

Other models like SVM could also be employed in a similar way. We will present the performance of the proposed features utilized in both two models.

6 EXPERIMENTAL RESULTS

In this section, we validate the effectiveness of the proposed approach.

6.1 Datasets

Data Preparation. We consider two data sets in our evaluation: Company and Product.

Company. Description of the company data set is given in Section 3. As there is no standard ground truth to quantitatively evaluate the performance of mining competitive relationships, for evaluation purpose, we have collected the competitive relationships between companies from *Yahoo! Finance*. Specifically, Yahoo! Finance provides a list of competitors for each company.[6] It also categorizes all the companies into different domains (called sector) such as technology, energy, and health. Each company may be classified into two domains. In this way, we create a

[6]For example, http://finance.yahoo.com/q/co?s=MSFT+Competitors.

ground truth for evaluating topic-level competitive relationships mining. In total, the company dataset contains 1,393 companies from three domains.

Product. The product data was extracted from Epinions, a website on which users pose reviews on their purchased products. We extracted information between two products such as price difference, reviewers who had reviewed on both of the products, comments that had both of the products' names as social networks features. The text information that supports the topic model was derived from the products' reviews. The dataset consists of 120 products, 972 reviews of the products, and 861 users who wrote comments on these products. Some example products include Canon 550D, Canon 5D Mark II (5d mii), Nikon D90, iPhone 4, iPad 2, and Amazon Kindle 2.

Evaluation. We conduct two types of experiments to evaluate the proposed approach. The first one is to identify global competitors. We evaluate the proposed model and compare it with alternative methods in terms of Precision (Prec.), Recall (Rec.), F1-Measure (F1), and Accuracy (Accu.). The second experiment is to detect competitors at specific topics, we define the probability of two competitors v_1 and v_2 competing in an area described by specific topic z as

$$p(v_1, v_2|z) = \frac{p(z|v_1)p(z|v_2)}{p(z)}. \tag{17}$$

In each experiment, we randomly picked 40% in each category as training (labeled) data and the rest as test (unlabeled) data. For evaluating the performance of topic-level competitor detection: We first determine whether two companies have a competitive relationship or not. After that, given a topic z and a company v_1, we rank its competitors by $p(v_1, v_2|z)$. At last we compare the rank with the ground truth from Yahoo! finance in terms of precision at position n (P@n), mean average precision (MAP), and normalized discount cumulative gain at position n (N@n). A similar method was previously used in [34].

We compare TFGM with the following baseline methods.

Content Similarity (CS). It calculates the cosine similarity between two companies' topic distributions and labels companies as competitors if their similarity value is greater than a threshold (0.2). We design it to see how unsupervised method works in this task.

Twitter Filtering (TF). It simply labels companies who have been mentioned in a same tweet at least one time as competitors. It is also an unsupervised method.

Random Walk with Restart (RW). It uses the network information to identify competitive relationships. Specifically, it builds up a tripartite graph which contains three types of node: inventors, companies, and patent categories (topics). For each company node v and topic node z, it creates a link from v to z and a link with opposite direction. Then, the random walk with restart algorithm [36, 37] is applied to rank competitors.

SVM. It uses all the features we defined in TFGM (see Appendix for details) to train a classification model (but SVM does not consider the correlation among the identified competitive relationships). We then employ it to predict the company pairs' labels in the test data. For SVM, we choose LIBSVM [3].

LR. It uses the same features as in the SVM method. The only difference is the way in which it uses logistic regression classification to predict the labels in the test data. The method was used in [21] to predict positive and negative links in social networks.

FGM. It trains a factor graph model with partially labeled data and all factors we defined in Section 4. This method can also been regarded as a special case of TFGM, when $\lambda = 0$. This method was used in [35] to classify the type of social relationships.

Table 1. Competitor Detection Performance of Different Methods in Three Domains

Domain	Method	Prec.	Rec.	F1	Accu.
.	CS	0.1858	0.7574	0.2983	0.3706
	TF	0.2312	0.2585	0.2441	0.5544
	RW	0.4605	0.2482	0.3226	0.8489
Tech	SVM	0.6643	0.5793	0.6189	0.8027
	LR	0.5636	0.5671	0.5653	0.7589
	FGM	0.7400	0.6768	0.7070	0.8449
	TFGM	**0.7576**	**0.7622**	**0.7599**	**0.8668**
	CS	0.2072	0.4200	0.2775	0.1335
	TF	0.3158	0.0882	0.1379	0.5930
	RW	0.3488	0.4115	0.3776	0.5774
Energy	SVM	0.4444	0.1429	0.2162	0.7844
	LR	0.3750	0.2143	0.2727	0.7621
	FGM	**0.6644**	0.9340	0.7765	**0.8571**
	TFGM	0.6558	**0.9528**	**0.7769**	0.8546
	CS	0.1175	0.0822	0.0967	0.0233
	TF	0.2727	0.0045	0.0089	0.5653
	RW	0.1581	0.1235	0.1387	0.6306
Health	SVM	0.7000	0.1000	0.1750	0.7471
	LR	0.1667	0.0142	0.0263	0.7165
	FGM	0.9041	0.9429	0.9231	0.9579
	TFGM	**0.9178**	**0.9571**	**0.9371**	**0.9655**

Bold face values stands for the best performance in the each task.

All algorithms are implemented in C++, and all experiments are performed on a Mac running Mac OS X with Intel Core i7 2.66 GHz and 4 GB memory. We empirically set the number of topics in TFGM as 100, and set parameters $\eta = 0.1$ and $\lambda = 0.5$ in all other experiments. We will give the sensitivity analysis of these parameters later. We also set the maximum iteration number $I = 500$ and $\mathcal{J} = 20$. In general, the efficiency of TFGM is acceptable. It takes 2 hours to learn from the company dataset.

6.2 Experimental Results

Table 1 shows the results of detect competitors globally with different approaches on the company dataset. We can see that TFGM clearly outperforms CS, TF, RW, SVM, and LR in all domains (+57.98% in terms of the average F1). CS, TF, and RW methods only consider content information, which leads to a bad performance. Compared with SVM and LR, one of TFGM's advantages is making use of the unlabeled data. Essentially, it further considers some latent correlations in the dataset, which cannot be leveraged with only the labeled training data. At the same time, TFGM also shows satisfying robustness. We can see that SVM and LR have unstable performances over different domains. For example, in Tech. domain, SVM has F1 of 0.62 that falls to 0.18 in Health domain. This is because competitive relationships in Health domain are quite sparse, which makes SVM mostly label company relationships as not competitive. Compared to FGM, with topic model incorporated, TFGM differentiates "competition topics" from those irrelevant topics and obtains a further improvement (e.g., +5% F1-score in Tech. domain).

Table 2. Topic-level Competitor Detection Performance of Different Methods in Three Domains

Domain	Method	P@5	P@10	MAP	N@5	N@10
Tech.	RW	0.3556	0.2616	0.3614	0.3917	0.3137
	TFGM	0.6762	0.4270	0.7657	0.6342	0.5542
Energy	RW	0.2455	0.1712	0.0518	0.2391	0.1898
	TFGM	0.6182	0.3614	0.8785	0.7079	0.6392
Health	RW	0.1067	0.1046	0.0094	0.1143	0.1104
	TFGM	0.3677	0.2225	0.8861	0.8233	0.7328

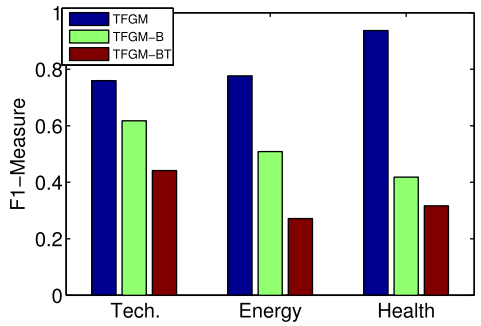


Fig. 7. Factors contribution. TFGM-B: ignoring balanced triangle factor function. TFGM-BT: further ignoring topic factor function.

There are two ways to detect topic-level competitors. One is the method we introduced above in Section 5.1. However, there is a different method for Random walk with restart: If we remove all "topic nodes" except one of them, the result would be the competitors in the corresponding topic. There are many ways in implementing the first method, e.g., all baselines. However, due to the space limitation, we only present results generated by TFGM. Also, baseline methods produced poor results in the first few steps; thus, it is reasonable to ignore them. Table 2 shows comparison result of TFGM and RW, from which we can see that TFGM clearly outperforms RW.

6.3 Analysis and Discussion

Factor Contribution. To determine the contributions of different factors to the model performance, we remove them one by one (first balanced triangle factor function, followed by the topic factor function), and then train and evaluate the performance. Figure 7 shows the F1-Measure score after ignoring the factor functions. We can observe clear drops on the performance, which indicates that each factor incorporated in the model has its specific contribution to the final result.

How Heterogeneous Networks Help. Social network and patent network are two fundamental constituent parts of the heterogeneous network we are studying on. To study how heterogeneous network helps solve this problem, we dismiss the two data source, respectively. Furthermore, we design another method to make use of the heterogeneous data source: we regard two companies as competitors if either of the methods based on a single data source labels them as competitors. Figure 8 shows the F1-Measure of these three methods comparing to the original approach. We can see that the model with both components incorporated exceeds the other two incomplete TFGM greatly in performance, which indicates that our model works better by learning across a heterogeneous network than either of the two networks. P+S's score drops greatly compared

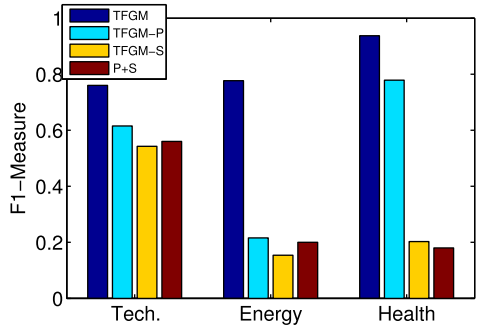
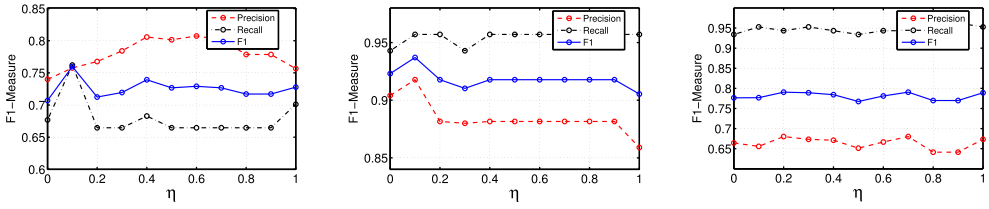


Fig. 8. Network contribution. TFGM-P: ignoring the patent information. TFGM-S: ignoring the social network. P+S: combining the two networks by simply labeling competitor relationships basing on whether TFGM-P or TFGM-S has labeled it.



(a) Tech. (b) Energy (c) Health

Fig. 9. Performance of TFGM by varying parameter η.

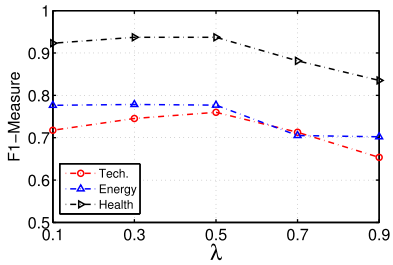


Fig. 10. Performance of TFGM by varying parameter λ.

with TFGM's. It even underperforms methods based on a single data source. By investigation, we find that if either one of TFGM-P and TFGM-S mistakenly labeled two entities as competitors, P+S keeps the mistake, which has severe adverse impact on the precision of the model.

Sensitivity Analysis. We conduct two experiments to test how parameter η and λ influence TFGM's performance. Figure 9 shows the trend of each measure following the changes of η in all domains (λ is fixed as 0.5). TFGM has low sensitivity of η in Energy and Health domains (the largest difference of F1 is less than 4% in both domains). However, in Tech. domain, the precision value slowly rises as η grows and then falls after $\eta = 0.6$. The recall value overall stays stable, yet it has a rapid fall from $\eta = 0.1$ to $\eta = 0.2$. We then fix $\eta = 0.1$ and see how F1-score changes by varying λ. As Figure 10 shows, the score increases slowly at the beginning but falls a bit more quickly when λ becomes larger (> 0.5).

Table 3. Examples of Topic-level Competitors

Topic	Words	Competitors
Topic #4	Image Graphics Pixel 3d	NVIDIA vs. Autodesk
		Adobe vs. VMware
		VMware vs. Autodesk
		Microsoft vs. NVIDIA
Topic #31	Database Distributed Query Domain	Oracle vs. Jabil Circuit
		Yahoo! vs. Jabil Circuit
		Google vs. Jabil Circuit
		Microsoft vs. Google
Topic #76	Semiconductor Toner Compositions Chamber	Novellus Systems vs. Intel
		First Solar vs. CREE
		Applied Materials vs. IBM
		Motorola vs. CREE

Table 4. Examples of Competitors Correlating Topics

Competitors	Topic	Hot words
Samsung vs. Apple	Topic #16	Communicating, microprocessors aluminum, cover, operating
	Topic #93	Electronic, processor, program memory, monitor, multiple
Microsoft vs. Google	Topic #48	Query, web, knowledge database, based, search
	Topic #81	Operating, program, service system, software, manage

6.3.1 Case Study. In this section, we demonstrate some examples generated from our experiments to show the effectiveness of our approach.

Topic-level Competitor Analysis. We study on topic-level competitor cases to see in real how text topics information helps competitor analysis. Table 3 displays results of several examples, listing the top competitors under the area given by a topic. As in Topic #4 describing graphic design, while the top competitors given by our model, NVIDIA and Autodesk, are two of the industry leaders.

On the other hand, given a pair of competitors, we try to figure out under which areas they are competing. Table 4 shows the top two topics for each pair of competitors according to $p(v_1, v_2|z)$. We can tell that our model finds Samsung and Apple actually correlating to topics like "communicating" and so on, indicating mobile phones, and "program," "processor," indicating computers—corresponds to the real situation. Similar results can be seen in topics correlated to Microsoft and Google.

Competitive Relationships between Products. Our model is flexible and can be easily applied to other datasets. We apply it to find competitive relationships between products. Here, the competitive relationships used to train our model are labeled by six human annotators. Two products are competitors to each other only if five of six annotators say so. Table 5 shows an example result compared with FGM. As we can see, both TFGM and FGM detect Nikon D90 as a competitor of both of the Canon cameras. But FGM wrongly labels Kindle 2 and 550D as competitors. Under our study, we find that many users discussed about how Kindle 2 or 550D is better than older versions, which makes the two products' distributions of the topic "version" similar to each other.

Table 5. Examples of Competitors Among Products. ✓: the results of TFGM, △: the results of FGM

	550d	5d mii	d90	Iphone4	Ipad2	kindle2
550d		✓ △	✓ △	✓		△
5d mii	✓ △		✓ △			
d90	✓ △	✓ △			△	
iphone4	✓					✓
ipad2			△			✓
kindle	△			✓	✓	

It, therefore, contributes a positive weight to labeling them as competitors. Yet, they are obviously not competitors and "version" is not a classic topic about competition. FGM is misled by this phenomenon while TFGM distinguishes this irrelevant topic from valuable ones.

Another interesting fact is that TFGM considers iPhone 4 and 550D as competitors. This is feasible since iPhone 4, with excellent photo-shooting performance and a similar price, is quite an alternative of 550D from customers' perspectives. At the same time, although iPad 2 has a camera built in, it is not often used for taking pictures. Thus, TFGM does not treat it as competitors of the cameras.

6.4 Results of Job-Hopping Prediction

To see how the competitive relationship mining problem helps the job-hopping prediction problem, we conduct another experiment based on the patent data set. For each pair of two companies (A, B), to define the order, we let A to be the one with more patents within time t comparing with B. We say an employee e hops to company B from company A at time t, if e applies patents affiliated in company A before time t, and applies patents affiliated in company B after time t. $l^{A,B} = 1$ $(A \rightarrow B)$ indicates that at time t there are more employees hopping to company B from A than employees hopping to company A from B. $l^{A,B} = -1$ $(A \leftarrow B)$ indicates the opponent case. $l^{A,B} = 0$ $(A \nleftrightarrow B)$ indicates that there is no employees hopping between company A and B at time t. In our experiment, we set t as the year 2010.

Table 6 presents the performance of job-hopping prediction. We present the prediction performance of all three labels and in average. LR and SVM are two methods utilizing the features we describe in Section 5 in Logistic Regression and SVM, respectively. LR-TFGM and SVM-TFGM are two baselines that utilizes all features but the one indicating whether A and B are competitors or not, which is calculated by TFGM.

From the results, we see clearly that knowing the competitive relationship between two companies helps the performance (e.g., +10.7% in term of F1 based on logistic regression). By a careful study, we find that without knowing whether the two companies are competitors or not, both LR and SVM tend to predict that there would be more employees hop to companies with more patents and inventors. However, they misjudge the case that there would be no job-hopping between companies with a great difference between their patents and inventors, but no business connections. The competitive relationships bring in the social information of companies and improves the recall.

7 RELATED WORK

In this section, we review the related works from four aspects: competitor detection, social tie mining, studies on Twitter, and patent mining.

Competitor Detection. Similar studies have been conducted with regards to competitor detection on the web. Using semantic analysis and text mining technique, Chen et al. [4] propose a

Table 6. Performance of Job-hopping Prediction

Subtasks	Method	Precision	Recall	F1-Measure
A → B	LR-TFGM	0.725	0.203	0.318
	LR	0.665	0.607	0.635
	SVM-TFGM	0.684	0.794	0.735
	SVM	0.775	0.905	0.835
A ← B	LR-TFGM	0.848	0.835	0.842
	LR	0.818	0.858	0.837
	SVM-TFGM	0.842	0.844	0.843
	SVM	0.842	0.844	0.843
A ↔ B	LR-TFGM	0.649	0.939	0.767
	LR	0.767	0.772	0.770
	SVM-TFGM	0.693	0.579	0.631
	SVM	0.992	0.821	0.899
Avg.	LR-TFGM	0.741	0.727	0.689
	LR	0.762	0.765	0.763
	SVM-TFGM	0.736	0.735	0.732
	SVM	0.870	0.858	0.859

framework to extract information from a user's website and learn his/her background knowledge. An algorithm that also infers competitive analysis is CoMiner, that Bao et al. [1] propose. CoMiner conducts a Web-scale mining for a company's competitive candidates, domain and competitive strength. Their methods, however, are significantly different from ours. We not only consider the text information, but also incorporate the social network information. Another related work is Liu et al.'s methods of discovering unexpected information from competitors' web sites [23]. This work focuses on analyzing competitors' features rather than detecting them, which is obviously different from what we are trying to do. Other related works including Li et al. [22] and Yang et al.'s [42] extract comparable entities by detecting keywords describing comparisons from online text documents. The two works study on a single data source, while our method utilizes heterogeneous networks.

In our previous work [43], we study the problem of competitive relationship mining in heterogeneous networks. In this article, we extend the previous work by considering another interesting problem: job-hopping prediction. We propose two categories of features and apply the results of previous work to solve the job-hopping prediction problem.

Social Tie Mining. Our work is related with social tie mining, which is an important problem in social network analysis. Several research branches existing in this topic. Diehl et al. [7] identify the manager–subordinate relationships by learning a ranking function. Wang et al. [38] propose an unsupervised probabilistic model for mining the advisor–advisee relationships from the publication network. Tang et al. [35] propose a semi-supervised probabilistic model to solve the problem of inferring social ties across social networks. Besides, Wang et al. [39] propose a new collaborative filtering approach to introduce a novel multiple-independent-Bernoulli-distribution model as the data sampling mask. Chen et al. [5] improve the prediction performance by constructing ensemble classifiers which are designed to preserve different measure of a network. A similar work is conducted by Zhang et al. [17].

Another branch is social behavior analysis. Tan et al. [31] investigate how social actions evolve in a dynamic social network. They propose a time varying factor graph model for modeling and predicting users' social behaviors. Tang et al. [33] study the difference of the social influence on

different topics and propose Topical Affinity Propagation (TAP) to model the topic-level social influence in social networks.

Twitter Study. Existing Twitter studies mainly include: Mathioudakis and Koudas [25] present a system, TwitterMonitor, to extract emerging topics from tweets' content; [13, 19, 31, 41] mainly focus on identifying influential users in Twitter or examining and predicting tweeting behaviors of users; Wei et al. [40] claim that Twitter based network structures are highly impacted by the removal of suspended users; Culotta et al. [6] predict Twitter users demographics based on the social ties between users; Kwak et al. [19] conduct a study on Twitter network and perceive some notable properties of Twitter; Hopcroft et al. [12] explore the problem of reciprocal relationship prediction on Twitter; Tang et al. [32] have developed a framework for classifying the type of social relationships by learning across heterogeneous networks. As far as we know, few works in the literature have tried to use Twitter or other microblog data for competitor detection.

Patent Mining. In this article, we also employ a set of patents information to assist for this competitor detections problem. There are also many related works on patent mining. Rodriguez et al. [29] provide a robust evaluation of the influence of patents by introducing a new kernel based influence measure in a patent citation network. Fu et al. [9] build an effective and automatic patent citation recommendation system. Kasravi et al. [16] propose a method to discover business value from patent repositories, Jin et al. [14] introduce a new problem of patent maintenance prediction and propose a method to solve it, while Ernst [8] uses patent information for strategic technology management including competitor monitoring. But these works only consider patent information, while we combine social networks and patents together to solve the competitor detection problem more effectively.

8 CONCLUSION AND FUTURE WORK

In this article, we study the problem of mining competitive relationships by learning across heterogeneous networks. Some features of competitive relationships, which reflect social network and patent information, are discovered and analyzed. We then formally define the problem in a semi-supervised framework and propose a TFGM for detecting competitors with social network and text document attributes given. In TFGM, factor graph and topic model are incorporated. Efficient algorithms are proposed for learning parameters as to infer unknown relationships. Experiments on two different datasets have been conducted and results outperform several alternatives greatly.

We then introduce the arrangement for our future work in short-term, mid-term, and long-term, respectively. The learning algorithm for the proposed TFGM model is a two-step method without a theoretical convenience guarantee. Thus in short-term, we aim to develop an uniform algorithm with a theoretical proof to learn the model better. Furthermore, sociological theory suggests that employees occupying structural hole positions in a competitive network control more access, timing, and referrals, and are key resources competitive companies aim to scramble. Enlightening by this theory, in mid-term, we would redefine the job-hopping prediction problem and study what roles structural hole spanners play in the competition between two companies. In long-term, we consider another interesting topic about how to detect potential collaborators. We believe that methods of collaborator analysis will resemble the ones that we propose in this article. In future work, we will try to apply the existing methods on competitive detection to collaborative detection and figure out whether additional theories or algorithms will need to be involved.

REFERENCES

[1] Shenghua Bao, Rui Li, Yong Yu, and Yunbo Cao. 2008. Competitor mining with the web. *IEEE Trans. Knowl. Data Eng.* 20 (2008), 1297–1310.

[2] David M. Blei, Andrew Y. Ng, and Michael I. Jordan. 2003. Latent Dirichlet allocation. *JMLR* 3 (2003), 993–1022.

[3] Chih-Chung Chang and Chih-Jen Lin. 2011. LIBSVM: A library for support vector machines. *ACM TIST* 2, 3 (2011), 27:1–27:27.
[4] Xin Chen and Yi-Fang Brook Wu. 2005. Web mining from competitors' websites. In *Proceedings of the 11th ACM SIGKDD International Conference on Knowledge Discovery in Data Mining (KDD'05)*. 550–555.
[5] Yi-Ling Chen, Ming-Syan Chen, and S. Yu Philip. 2015. Ensemble of diverse sparsifications for link prediction in large-scale networks. In *Proceedings of the 2015 IEEE International Conference on Data Mining (ICDM)*. IEEE, 51–60.
[6] Aron Culotta, Nirmal Ravi Kumar, and Jennifer Cutler. 2015. Predicting the demographics of twitter users from website traffic data. In *Proceedings of AAAI*. 72–78.
[7] Christopher P. Diehl, Galileo Namata, and Lise Getoor. 2007. Relationship identification for social network discovery. In *Proceedings Of The National Conference On Artificial Intelligence*. Vol. 22. AAAI Press; MIT Press, Menlo Park, CA; Cambridge, MA, London, 1999, 546.
[8] Holger Ernst. 2003. Patent information for strategic technology management. *World Patent Information* 25, 3 (September 2003), 233–242.
[9] Tao-yang Fu, Zhen Lei, and Wang-Chien Lee. 2015. Patent citation recommendation for examiners. In *Proceedings of the 2015 IEEE International Conference on Data Mining (ICDM)*. IEEE, 751–756.
[10] F. Heider. 1946. Attitudes and cognitive organization. *Journal of Psychology* 21, 2 (1946), 107–112.
[11] Thomas Hofmann. 1999. Probabilistic latent semantic indexing. In *Proceedings of SIGIR'99*. 50–57.
[12] John E. Hopcroft, Tiancheng Lou, and Jie Tang. 2011. Who will follow you back? Reciprocal relationship prediction. In *Proceedings of CIKM'11*.
[13] Bernardo A. Huberman, Daniel M. Romero, and Fang Wu. 2009. Social networks that matter: Twitter under the microscope. *First Monday* 14, 1 (2009), 1–5.
[14] Xin Jin, Scott Spangler, Ying Chen, Keke Cai, Rui Ma, Li Zhang, Xian Wu, and Jiawei Han. 2011. Patent maintenance recommendation with patent information network model. In *Proceedings of ICDM'11*.
[15] Jalil Khavand Kar and Ehsan Khavandkar. 2013. *Intellectual Capital: Managing Development and Measurement Models*. MSRT.
[16] Kas Kasravi and Maria Risov. 2007. Patent mining - Discovery of business value from patent repositories. In *Proceedings of Hawaii International Conference on System Sciences*. 54b.
[17] Angelos Kremyzas, Norman Jaklin, and Roland Geraerts. 2016. Towards social behavior in virtual-agent navigation. *Sci. China Inform. Sci.* 59, 11 (2016), 112102.
[18] Frank R. Kschischang, Brendan J. Frey, and Hans Andrea Loeliger. 2001. Factor graphs and the sum-product algorithm. *IEEE TOIT* 47, 2 (2001), 498–519.
[19] Haewoon Kwak, Changhyun Lee, Hosung Park, and Sue B. Moon. 2010. What is twitter, a social network or a news media? In *Proceedings of WWW*. 591–600.
[20] Paul F. Lazarsfeld and Robert K. Merton. 1954. *Friendship as a Social Process: A Substantive and Methodological Analysis*. Vol. 18. Van Nostrand, 18–66.
[21] Jure Leskovec, Daniel P. Huttenlocher, and Jon M. Kleinberg. 2010. Predicting positive and negative links in online social networks. In *Proceedings of WWW'10*. 641–650.
[22] Shasha Li, Chin-Yew Lin, Young-In Song, and Zhoujun Li. 2010. Comparable entity mining from comparative questions. In *Proceedings of the 48th AMACL (ACL'10)*. 650–658.
[23] Bing Liu, Yiming Ma, and Philip S. Yu. 2001. Discovering unexpected information from your competitors' web sites. In *Proceedings of KDD*. 144–153.
[24] J. Maddocks and M. Beaney. 2002. See the invisible and intangible. *Knowl. Manag.* 3 (2002), 16–17.
[25] Michael Mathioudakis and Nick Koudas. 2010. TwitterMonitor: Trend detection over the twitter stream. In *Proceedings of SIGMOD'10*. 1155–1158.
[26] Qiaozhu Mei, Deng Cai, Duo Zhang, and ChengXiang Zhai. 2008. Topic modeling with network regularization. In *Proceedings of WWW'08*. 101–110.
[27] Kevin P. Murphy, Yair Weiss, and Michael I. Jordan. 1999. Loopy belief propagation for approximate inference: An empirical study. In *Proceedings of UAI'99*. 467–475.
[28] Michael E. Porter. 1998. *Competitive Strategy: Techniques for Analyzing Industries and Competitors* (1 ed.). Free Press.
[29] Andrew Rodriguez, Byunghoon Kim, Jae-Min Lee, Byoung-Yul Coh, and Myong K. Jeong. 2015. Graph kernel based measure for evaluating the influence of patents in a patent citation network. *Expert Syst. Appl.* 42, 3 (2015), 1479–1486.
[30] Jian-Tao Sun, Xuanhui Wang, Dou Shen, Hua-Jun Zeng, and Zheng Chen. 2006. CWS: A comparative web search system. In *Proceedings of WWW'06*. 467–476.
[31] Chenhao Tan, Jie Tang, Jimeng Sun, Quan Lin, and Fengjiao Wang. 2010. Social action tracking via noise tolerant time-varying factor graphs. In *Proceedings of KDD'10*. 1049–1058.
[32] Jie Tang, Tiancheng Lou, and Jon Kleinberg. 2012. Inferring social ties across heterogenous networks. In *Proceedings of WSDM'12*. 743–752.

[33] Jie Tang, Jimeng Sun, Chi Wang, and Zi Yang. 2009. Social influence analysis in large-scale networks. In *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 807–816.
[34] Jie Tang, Bo Wang, Yang Yang, Po Hu, Yanting Zhao, Xinyu Yan, Bo Gao, Minlie Huang, Peng Xu, Weichang Li, and Adam K. Usadi. 2012. PatentMiner: Topic-driven patent analysis and mining. In *Proceedings of KDD'2012*.
[35] Wenbin Tang, Honglei Zhuang, and Jie Tang. 2011. Learning to infer social ties in large networks. In *Proceedings of ECML/PKDD (3)'11*. 381–397.
[36] Hanghang Tong, Christos Faloutsos, and Yehuda Koren. 2007. Fast direction-aware proximity for graph mining. In *Proceedings of KDD'07*. 747–756.
[37] Hanghang Tong, Christos Faloutsos, and Jia-Yu Pan. 2006. Fast random walk with restart and its applications. In *Proceedings of ICDM'06*. 613–622.
[38] Chi Wang, Jiawei Han, Yuntao Jia, Jie Tang, Duo Zhang, Yintao Yu, and Jingyi Guo. 2010. Mining advisor-advisee relationships from research publication networks. In *Proceedings of KDD'10*. ACM, 203–212.
[39] Xiangyu Wang, Dayu He, Danyang Chen, and Jinhui Xu. 2015. Clustering-based collaborative filtering for link prediction. In *Proceedings of AAAI*. 332–338.
[40] Wei Wei, Kenneth Joseph, Huan Liu, and Kathleen M. Carley. 2016. Exploring characteristics of suspended users and network stability on Twitter. *Social Network Analysis and Mining* 6, 1 (2016), 51.
[41] Jianshu Weng, Ee-Peng Lim, Jing Jiang, and Qi He. 2010. TwitterRank: Finding topic-sensitive influential twitterers. In *Proceedings of WSDM*, Brian D. Davison, Torsten Suel, Nick Craswell, and Bing Liu (Eds.). 261–270.
[42] Seon Yang and Youngjoong Ko. 2011. Extracting comparative entities and predicates from texts using comparative type classification. In *Proceedings of the 49th AMACL (HLT'11)*. 1636–1644.
[43] Yang Yang, Jie Tang, Jacklyne Keomany, Yanting Zhao, Juanzi Li, Ying Ding, Tian Li, and Liangwei Wang. 2012. Mining competitive relationships by learning across heterogeneous networks. In *Proceedings of CIKM'12*. ACM, 1432–1441.
[44] Xiaojin Zhu and John Lafferty. 2005. Harmonic mixtures: Combining mixture models and graph-based methods for inductive and scalable semi-supervised learning. In *Proceedings of ICML'05*. 1052–1059.

Received September 2015; revised July 2016; accepted February 2017